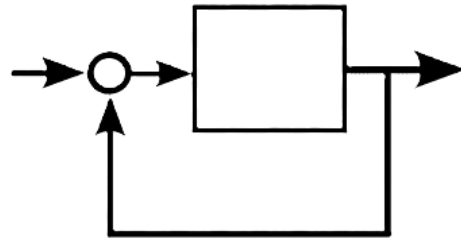


AA 447: Control in Aerospace Systems



Today's plan

1. Matlab:

1. P
2. PI
3. PID

Matlab code

- Starter code on Canvas under Files
 - Defining transfer functions
 - Plotting responses to step & arbitrary inputs
 - Plotting Bode plots
 - Plotting Pole/Zero maps

Canvas->Files->AA447-Matlab-intro.pdf

Controls in Matlab - Getting started

Creating Transfer Functions

Option 1: "tf"

Define coefficients to polynomial in numerator and denominator

Ex: $G = s / (s^2 + 3s + 2)$

```
num = [1 0];
```

```
den = [1 3 2];
```

```
G = tf(num,den);
```

Option 2: "zpk"

Define Zeros (Z), Poles (P), and gain (K) explicitly

Ex: $G = 5 s / ((s+1+i) (s+1-i) (s+2))$

```
Z = [0];
```

```
P = [-1-1i -1+1i -2];
```

```
K = 5;
```

```
G = zpk(Z,P,K);
```

System Characteristics

Step response

Plot step response of system `sys` from $U=0$ to $U=1$. You can also specify initial conditions (default = 0), set simulation time, etc. See documentation.

```
step(sys)
```

Matlab code

Create transfer functions with tf()

- Define coefficients to polynomial in numerator & denominator

Ex:

$$G = s / (s^2 + 3s + 2)$$

```
num = [1 0];
```

```
den = [1 3 2];
```

```
G = tf(num,den);
```

Matlab code

Create transfer functions with tf()

- Define coefficients to polynomial in numerator & denominator

Ex:

$$G = s / (s^2 + 3s + 2)$$

```
num = [1 0];
```

```
den = [1 3 2];
```

```
G = tf(num,den);
```

```
G = tf([1 0],[1,3,2])
```

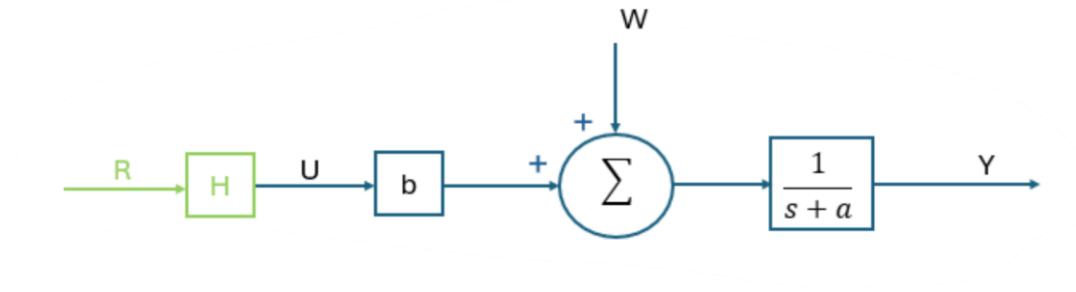
G =

$$\frac{s}{s^2 + 3s + 2}$$

P Control

Proportional Control

Open Loop ($H=k_p$)

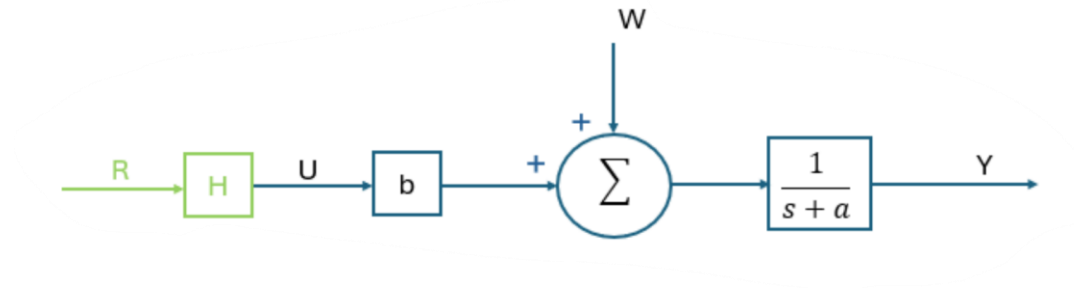


$$G_{RY} = \frac{Hb}{s+a}$$

$$G_{wY} = \frac{1}{s+a}$$

Proportional Control

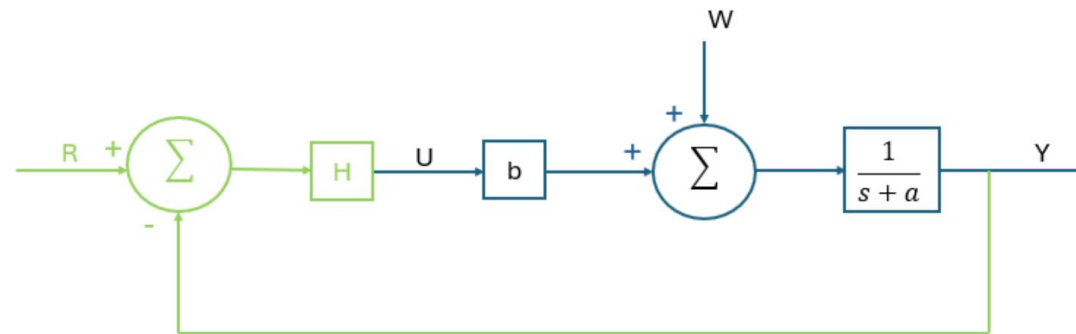
Open Loop ($H=k_p$)



$$G_{RY} = \frac{Hb}{s+a}$$

$$G_{wY} = \frac{1}{s+a}$$

Closed Loop ($H=k_p$)



$$G_{RY} = \frac{Hb}{s+a+Hb}$$

$$G_{wY} = \frac{1}{s+a+Hb}$$

Prop. Control – Tracking (step)

$$\text{Open Loop (H=k}_p\text{)} \quad G_{RY} = \frac{Hb}{s + a}$$

$$\text{Closed Loop (H=k}_p\text{)} \quad G_{RY} = \frac{Hb}{s + a + Hb}$$

```
% Set variables for our pole a, input multiplier b, gain kp
a = 1;
b = 1;
kp_ol = a/b;
kp_cl = 10;

% Define Gry - Open and closed loop
G_ry_ol = tf(kp_ol*b,[1 a]) % in OL, we set Kp=a/b -> a/(s+a)
G_ry_cl = tf((kp_cl*b),[1 (kp_cl*b+a)]) % kp*b/(s+kp*b+a)
```

```
% Gry - Step Response
step(G_ry_ol,G_ry_cl)
legend('G_ry_ol','G_ry_cl')
title('Step response (Gry) - Open vs Closed Loop')
```

Prop. Control – Tracking (step)

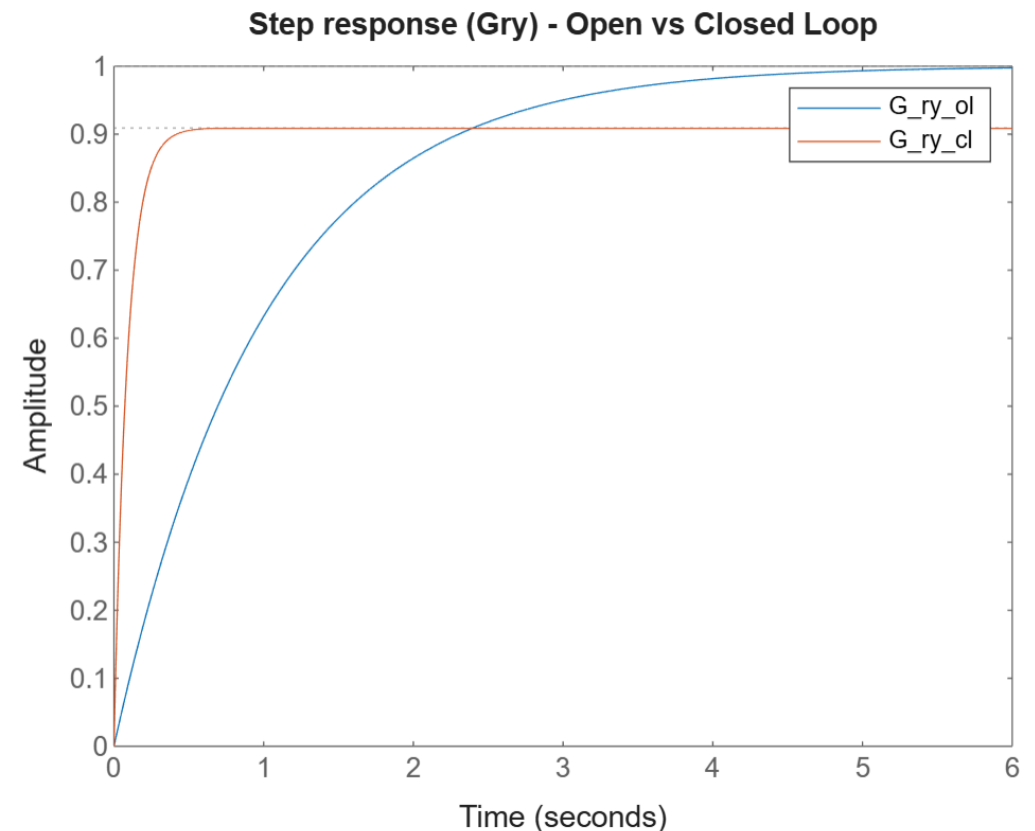
$$\text{Open Loop (H=k}_p\text{)} \quad G_{RY} = \frac{Hb}{s + a}$$

$$\text{Closed Loop (H=k}_p\text{)} \quad G_{RY} = \frac{Hb}{s + a + Hb}$$

```
% Set variables for our pole a, input multiplier b, gain kp
a = 1;
b = 1;
kp_ol = a/b;
kp_cl = 10;

% Define Gry - Open and closed loop
G_ry_ol = tf(kp_ol*b,[1 a]) % in OL, we set Kp=a/b -> a/(s+a)
G_ry_cl = tf((kp_cl*b),[1 (kp_cl*b+a)]) % kp*b/(s+kp*b+a)
```

```
% Gry - Step Response
step(G_ry_ol,G_ry_cl)
legend('G_ry_ol','G_ry_cl')
title('Step response (Gry) - Open vs Closed Loop')
```

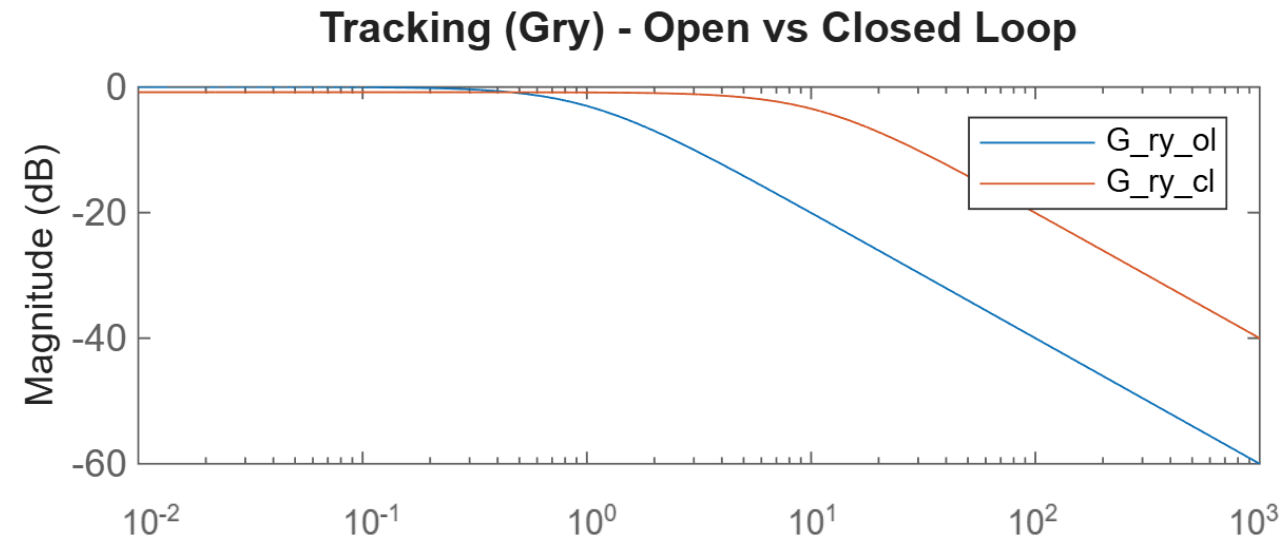


Prop. Control – Tracking (bode)

$$\text{Open Loop (H=k}_p\text{)} \quad G_{RY} = \frac{Hb}{s + a}$$

$$\text{Closed Loop (H=k}_p\text{)} \quad G_{RY} = \frac{Hb}{s + a + Hb}$$

```
% Bode Plot - OL/CL Gry comparison  
bode(G_ry_ol,G_ry_cl)  
legend('G_ry_ol','G_ry_cl')  
title('Tracking (Gry) - Open vs Closed Loop')
```



Prop. Control – Disturbance rejection (bode)

Open Loop

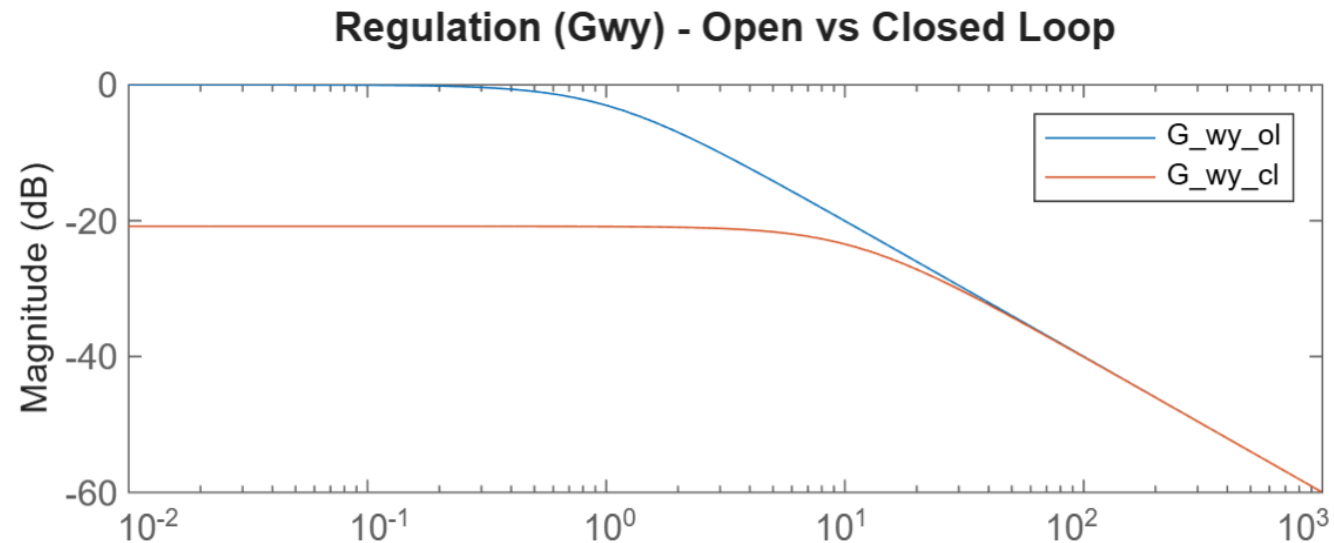
$$G_{WY} = \frac{1}{s + a}$$

Closed Loop ($H=k_p$)

$$G_{WY} = \frac{1}{s + a + Hb}$$

```
% Define Gwy - Open and closed loop
G_wy_ol = tf(1,[1 a]) % 1/(s+a)
G_wy_cl = tf(1,[1 (kp_cl*b+a)]) % 1/(s+kp*b+a)

% Bode Plot - OL/CL Gwy comparison
bode(G_wy_ol,G_wy_cl)
legend('G_wy_ol','G_wy_cl')
title('Regulation (Gwy) - Open vs Closed Loop')
```



Prop. Control – Tracking with disturbance

$$\text{Open Loop} \quad G_{RY} = \frac{Hb}{s+a} \quad G_{WY} = \frac{1}{s+a}$$

$$\text{Closed Loop} \quad G_{RY} = \frac{Hb}{s+a+Hb} \quad G_{WY} = \frac{1}{s+a+Hb}$$

```
% Response to step input and sinusoidal disturbance at freq a
% (CL attenuates disturbance freq a by ~1/(kp*b+a), OL by ~1/a)
t = 0:0.05:40;
u = ones(size(t));
w = 0.1*sin(a*t);

% Open loop Y = Gry*U + Gwy*d- Step Input and sinusoidal disturbance
y1_ol = lsim(G_ry_ol,u,t);
y2_ol = lsim(G_wy_ol,w,t);
y_ol = y1_ol + y2_ol;
plot(t,y_ol)
title('Open vs Closed Loop: Step response + sinusoidal disturbance')
hold on

% Closed loop Y = Gry*U + Gwy*d- Step Input and sinusoidal disturbance
y1_cl = lsim(G_ry_cl,u,t);
y2_cl = lsim(G_wy_cl,w,t);
y_cl = y1_cl + y2_cl;
plot(t,y_cl)
legend('Open loop','Closed loop')
hold off
```

Prop. Control – Tracking with disturbance

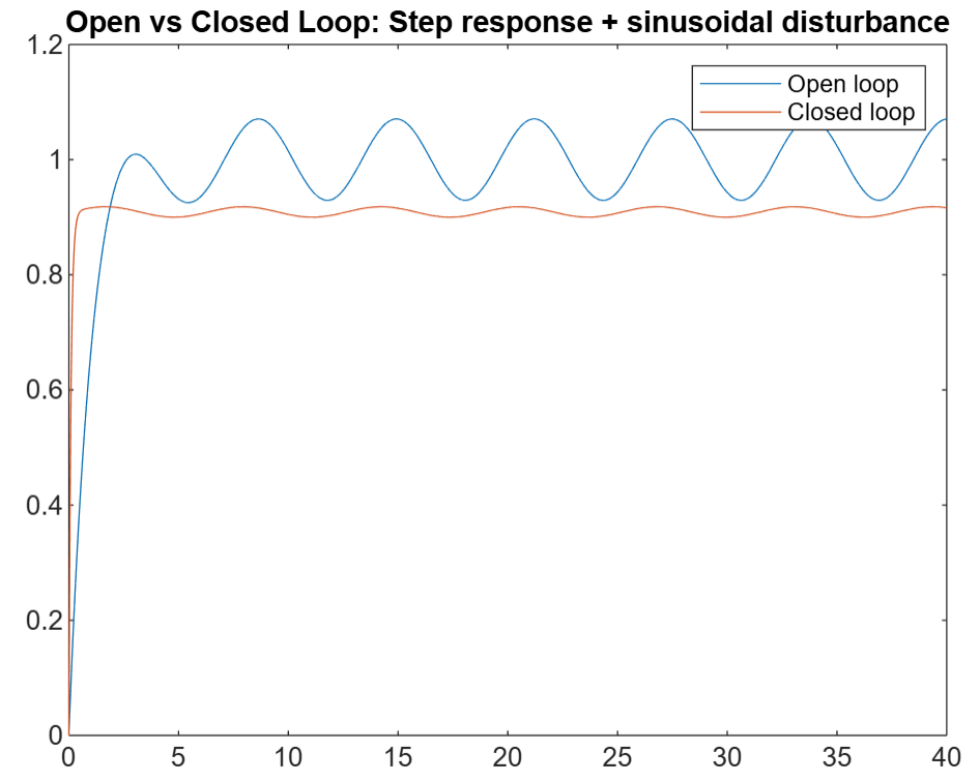
Open Loop $G_{RY} = \frac{Hb}{s+a}$ $G_{WY} = \frac{1}{s+a}$

Closed Loop $G_{RY} = \frac{Hb}{s+a+Hb}$ $G_{WY} = \frac{1}{s+a+Hb}$

```
% Response to step input and sinusoidal disturbance at freq a
% (CL attenuates disturbance freq a by ~1/(kp*b+a), OL by ~1/a)
t = 0:0.05:40;
u = ones(size(t));
w = 0.1*sin(a*t);

% Open loop Y = Gry*U + Gwy*d- Step Input and sinusoidal disturbance
y1_ol = lsim(G_ry_ol,u,t);
y2_ol = lsim(G_wy_ol,w,t);
y_ol = y1_ol + y2_ol;
plot(t,y_ol)
title('Open vs Closed Loop: Step response + sinusoidal disturbance')
hold on

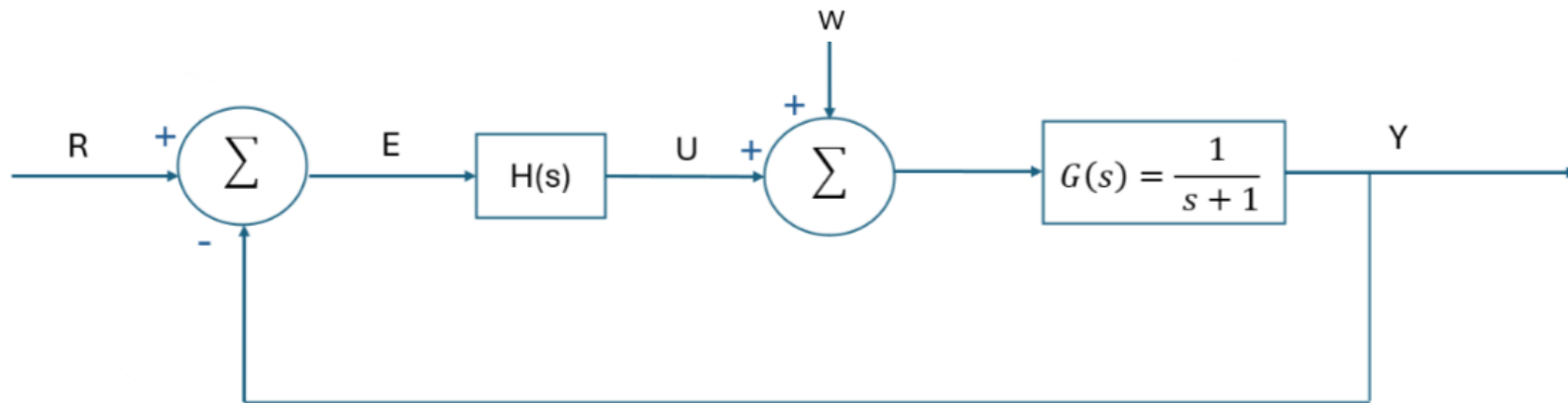
% Closed loop Y = Gry*U + Gwy*d- Step Input and sinusoidal disturbance
y1_cl = lsim(G_ry_cl,u,t);
y2_cl = lsim(G_wy_cl,w,t);
y_cl = y1_cl + y2_cl;
plot(t,y_cl)
legend('Open loop','Closed loop')
hold off
```



PI Control

PI Control

Controller $H = k_p + k_I/s$



$$G_{RY} = \frac{K_p s + K_I}{s^2 + (K_p + 1)s + K_I}$$

$$G_{WY} = \frac{s}{s^2 + (K_p + 1)s + K_I}$$

PI Control – Tracking (step)

Effect on tracking of varying K_p for constant K_I

Closed Loop ($H = k_p + K_I/s$)

$$G_{RY} = \frac{K_p s + K_I}{s^2 + (K_p + 1)s + K_I}$$

```
% Set variables
a = 1;
ki = 1;
kp_values = [0.1,1,10];

figure;
hold on;
legend_labels = {};
for kp = kp_values
    % Define transfer functions
    G = tf(1, [1 a]); % our plant
    H = tf([kp ki], [1 0]); % PI controller
    GH = G*H; % open loop TF
    Gry = feedback(GH,1) % closed loop TF: GH/(1+GH)
    step(Gry)
    legend_labels{end+1} = strcat('kp=', num2str(kp)); % Append label to legend
end
legend(legend_labels, 'Location', 'southeast');
title('Reference Input Step response; ki=1, varying kp')
% Larger kp makes system faster (decreasing rise time), and can reduce
% oscillation by causing roots to become real (b^2-4ac becomes positive)
```

PI Control – Tracking (step)

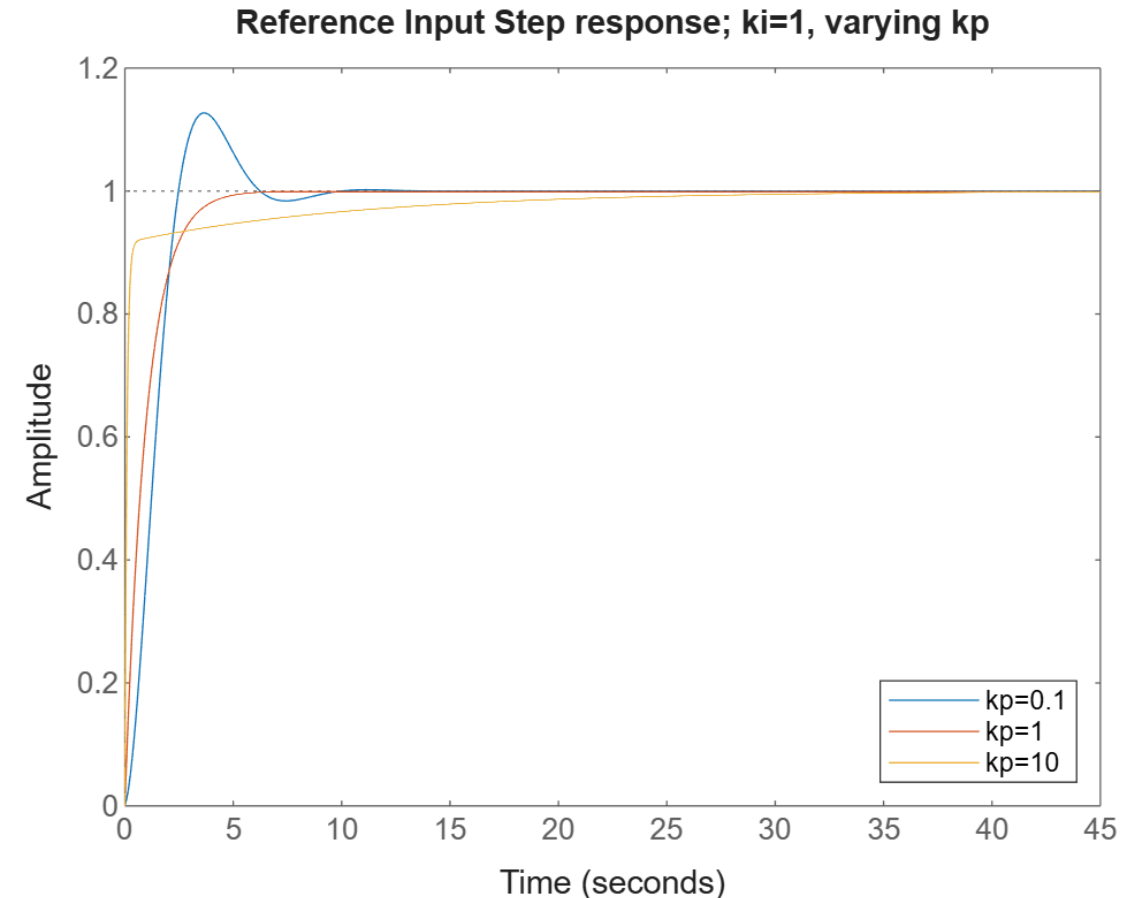
Effect on tracking of varying K_p for constant K_I

Closed Loop ($H = k_p + K_I/s$)

$$G_{RY} = \frac{K_p s + K_I}{s^2 + (K_p + 1)s + K_I}$$

```
% Set variables
a = 1;
ki = 1;
kp_values = [0.1,1,10];

figure;
hold on;
legend_labels = {};
for kp = kp_values
    % Define transfer functions
    G = tf(1, [1 a]); % our plant
    H = tf([kp ki], [1 0]); % PI controller
    GH = G*H; % open loop TF
    Gry = feedback(GH,1) % closed loop TF: GH/(1+GH)
    step(Gry)
    legend_labels{end+1} = strcat('kp=', num2str(kp)); % Append label to legend
end
legend(legend_labels, 'Location', 'southeast');
title('Reference Input Step response; ki=1, varying kp')
% Larger kp makes system faster (decreasing rise time), and can reduce
% oscillation by causing roots to become real (b^2-4ac becomes positive)
```



PI Control – Tracking (step)

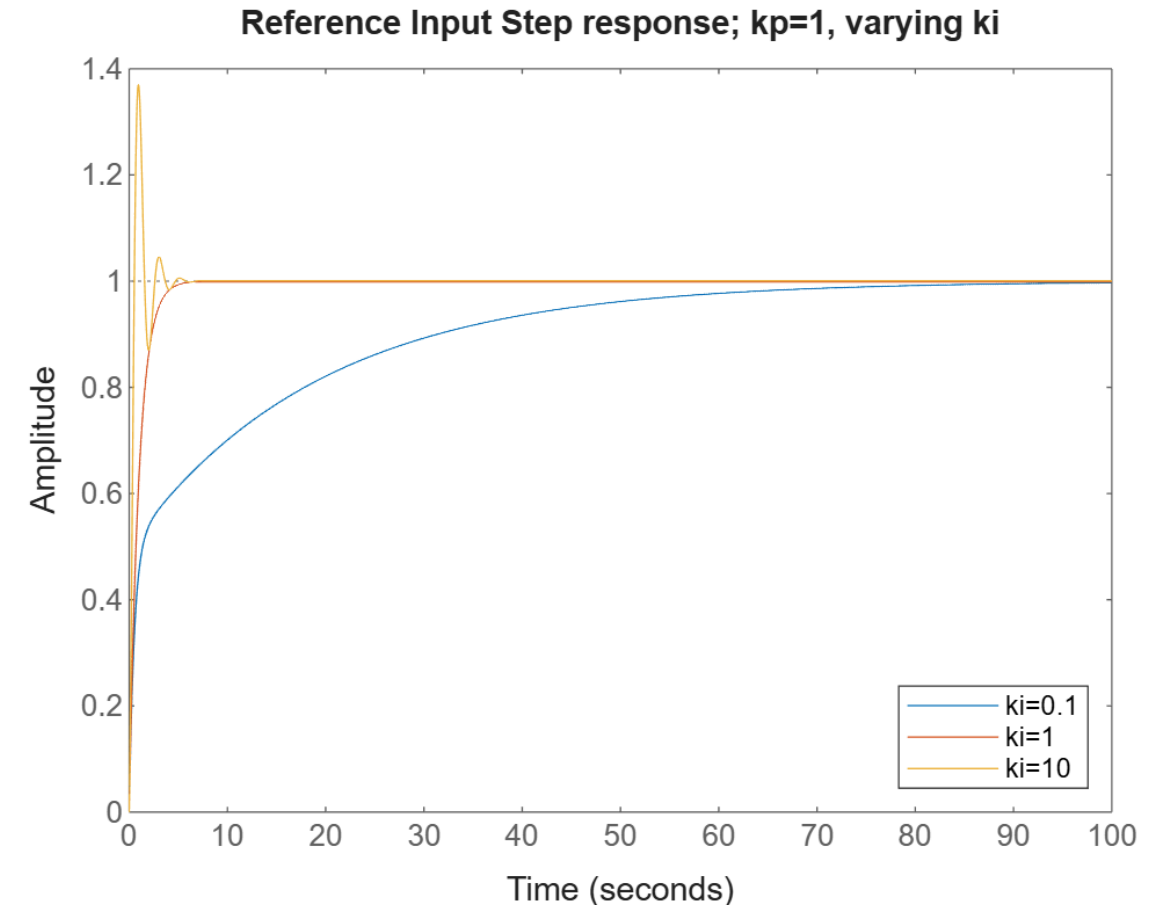
Effect on tracking of varying K_I for constant K_P

Closed Loop ($H = k_p + K_I/s$)

$$G_{RY} = \frac{K_p s + K_I}{s^2 + (K_p + 1)s + K_I}$$

```
% Set variables
a = 1;
kp = 1;
ki_values = [0.1,1,10];

figure;
hold on;
legend_labels = {};
for ki = ki_values
    % Define transfer functions
    G = tf(1, [1 a]); % our plant
    H = tf([kp ki], [1 0]); % PI controller
    GH = G*H; % open loop TF
    Gry = feedback(GH,1) % closed loop TF: GH/(1+GH)
    step(Gry)
    legend_labels{end+1} = strcat('ki=', num2str(ki));
end
legend(legend_labels, 'Location', 'southeast');
title('Reference Input Step response; kp=1, varying ki')
% Small ki still gets us 0 steady-state error, but integrates
% slowly, taking long time to reach the setpoint.
% Large ki causes "windup"- it integrates too much error, causing overshoot.
```

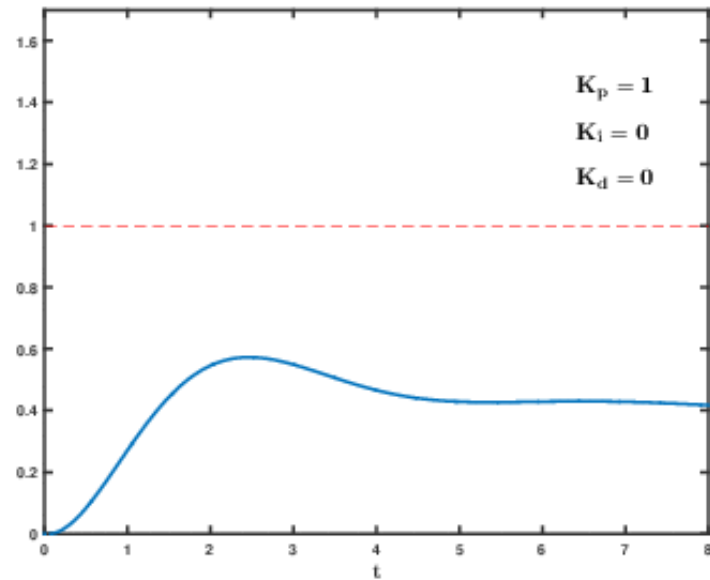


PID Control

PID Control

Controller Tuning:

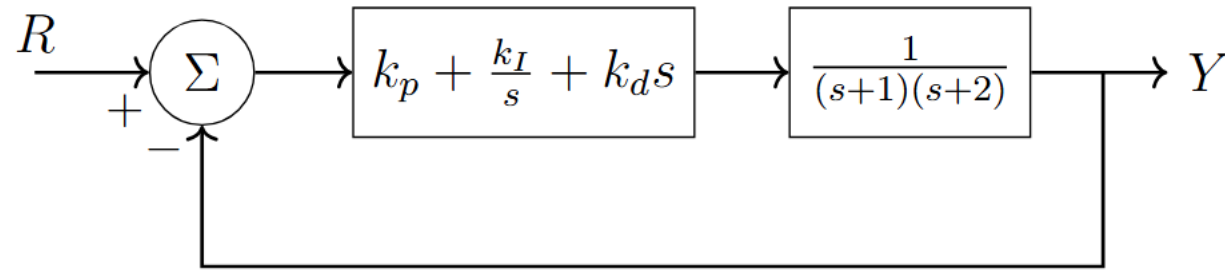
- 1) Increase P (reduce error quickly)
- 2) Increase I (eliminate steady-state error)
- 3) Increase D (improve performance)



PID Control

Controller $H = k_p + k_I/s + k_d s$

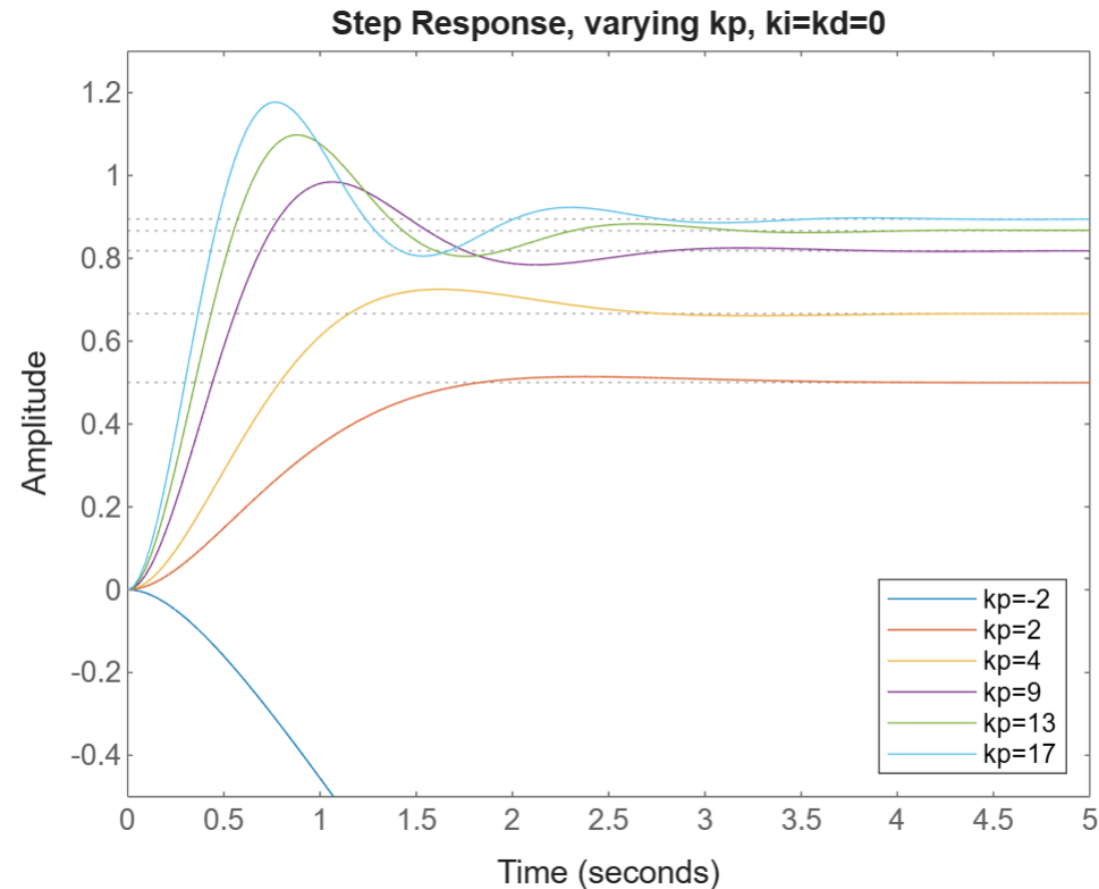
Plant $G = 1/(s+1)(s+2)$



$$G_{RY}(s) = \frac{K_d s^2 + K_p s + K_i}{s^3 + (3 + K_d)s^2 + (2 + K_p)s + K_i}$$

PID Control – Increase P

Step response: increasing K_p , while $K_I = K_d = 0$



PID example code

Note: Matlab minreal()
- cancels zero/pole pairs

Before minreal():

$$\text{Gry} = \frac{2s}{s(s^2 + 3s + 4)}$$

After minreal():

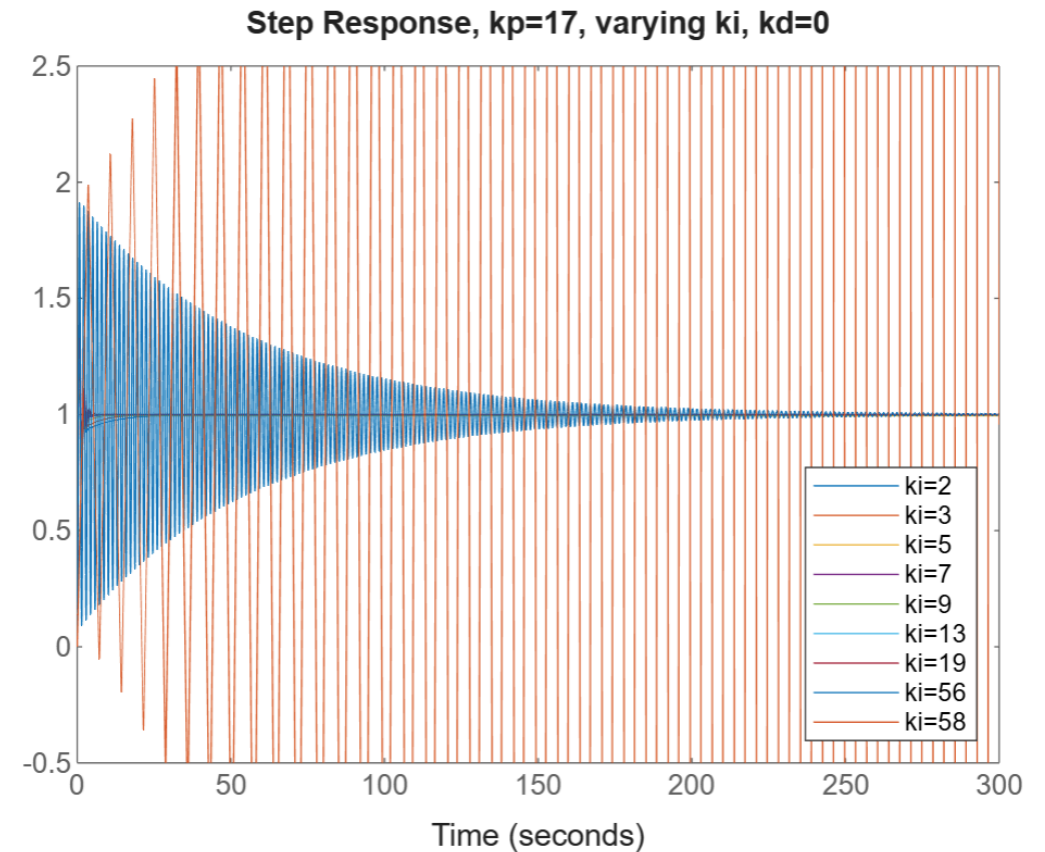
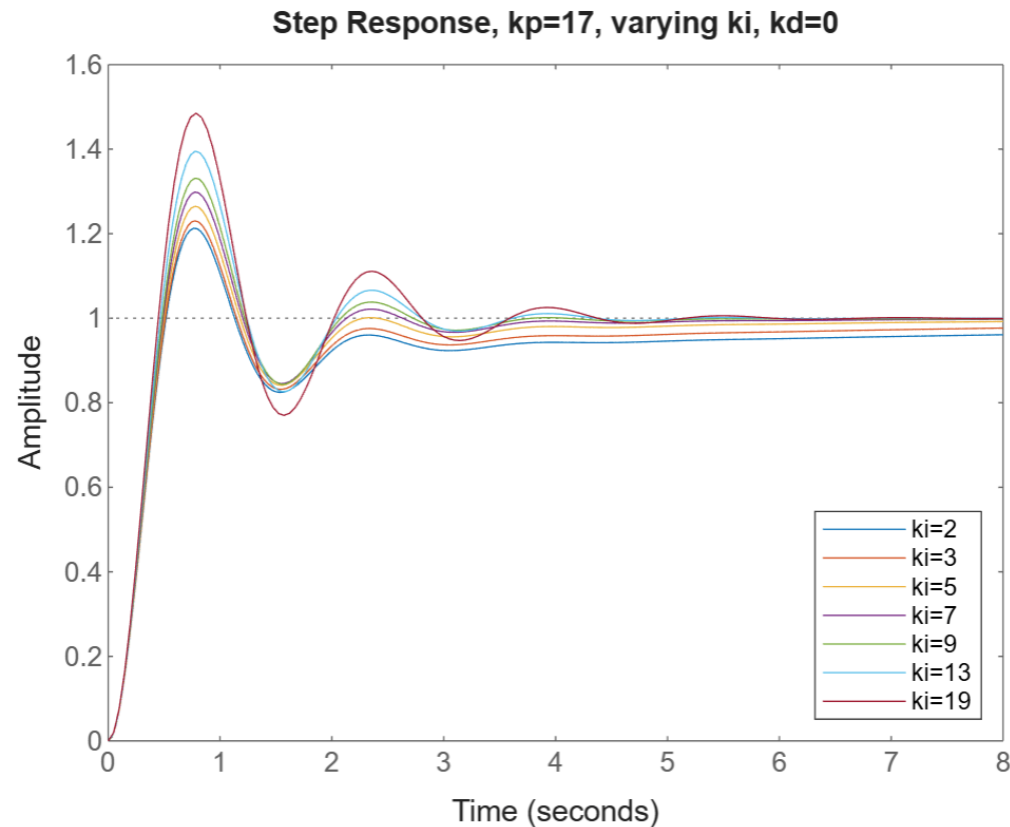
$$\text{Gry} = \frac{2}{(s^2 + 3s + 4)}$$

```
%% Increase P
% Set variables
a = -2;
b = -1;
kp_values = [-2,2,4,9,13,17];
ki = 0;
kd = 0;

figure;
hold on;
legend_labels = {};
for kp = kp_values
    % Plant G = 1/(s+1)(s+2)
    Z=[];
    P=[a b]; %
    K= 1;
    G = zpk(Z,P,K);
    % PID Controller H
    H = tf([kd kp ki], [1 0]); % PID controller
    GH = G*H; % open loop TF
    Gry = feedback(GH,1) % closed loop TF: GH/(1+GH)
    Gry = minreal(Gry)
    step(Gry)
    legend_labels{end+1} = strcat('kp=', num2str(kp));
end
legend(legend_labels, 'Location', 'southeast');
title('Step Response, varying kp, ki=kd=0')
```

PID Control – Fix P, Increase I

Step response: increasing K_I , while $K_p = 17$, $K_d = 0$



PID Control – Fix P,I, Increase D

Step response: increasing K_d , while $K_p = 17$, $K_I = 13$

